



Atividades - 3: Comunicação entre tarefas

Professor: Rogério Pereira Junior

rogerio.pereira@ifsc.edu.br

Objetivo da Atividade

- O objetivo desta pesquisa é identificar a aplicação prática dos conceitos de Comunicação entre Processos (IPC) em arquiteturas de software modernas.
- Os alunos deverão conectar os termos teóricos (sincronismo, buffering, confiabilidade e endereçamento) com tecnologias utilizadas no mercado.

Orientações Gerais

A turma deve ser dividida em 4-3 grupos.

Cada grupo será responsável por um tema específico.

A entrega final consistirá em uma **apresentação curta (5 min)** e um **Diagrama de Fluxo de Mensagem**.

Temas de Pesquisa

Grupo 1: Interfaces e Desktop (O Modelo D-Bus)

Como o sistema operacional comunica eventos de hardware (bateria fraca, USB conectado) para as aplicações do usuário?

- Pesquisar: O que é o modelo *Publish-Subscribe*?
- Diferenciar o *System Bus* do *Session Bus*.
- O D-Bus é síncrono ou assíncrono? Justifique.

Grupo 2: Jogos Online e Latência (Sockets UDP vs. TCP)

Em jogos competitivos, a velocidade é mais crítica que a integridade total dos dados.

- Por que o protocolo TCP pode causar o fenômeno de "lag" em jogos?
- Como o UDP lida com a falta de confiabilidade e ordenação?
- TCP e UDP qual o formato de envio? Fluxo ou Mensagens?
- Pesquisar: Como os desenvolvedores usam a "Predição do Lado do Cliente" para mascarar a latência?

Grupo 3: Microserviços e Escala (RabbitMQ ou Apache Kafka)

Como sistemas globais (Netflix, Amazon) garantem que mensagens de pagamento não se percam se um servidor falhar?

- O que é um *Message Broker* e como ele provê capacidade de buffer infinita (persistência)?
- Analisar a vantagem da comunicação indireta (mailboxes) para escalabilidade.
- O que acontece quando o produtor é mais rápido que o consumidor nestes sistemas?

Grupo 4: Controle e Notificação (Sinais do Sistema)

Nem toda comunicação entre tarefas envolve a troca de grandes volumes de dados. Muitas vezes, um processo precisa enviar alertas ou comandos de controle para outro, como interrupções ou avisos de finalização. No Linux, isso é realizado através de **Sinais (Signals)**.

- **Natureza do Sinal:** Explique a diferença entre enviar um dado via Pipe e enviar um sinal de interrupção. O sinal transporta conteúdo ou apenas uma notificação?
- **Gestão de Processos:** Pesquise o papel dos sinais *SIGINT*, *SIGKILL* e *SIGTERM*. Como eles são usados no dia a dia por administradores de sistema?
- **Reação a Eventos:** O que é um "tratador de sinal" (*signal handler*) e por que ele é importante para garantir que um programa feche de forma segura, sem corromper dados?

O que deve constar no Diagrama de Fluxo

O diagrama deve ilustrar um cenário real do tema escolhido, identificando:

1. **O Emissor:** Quem gera a informação.
2. **O Canal:** O meio de transporte (Socket, Pipe, Shared Memory, etc).
3. **O Receptor:** Quem processa a informação.
4. **Propriedades:** Indicar se é Direta/Indireta, Síncrono/Assíncrono, Por mensagens ou fluxos e Confiável/Não-Confiável.